

Data Handling & Visualization

KAUST–Mawhiba IOAI Training

Week 1 · Day 2



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

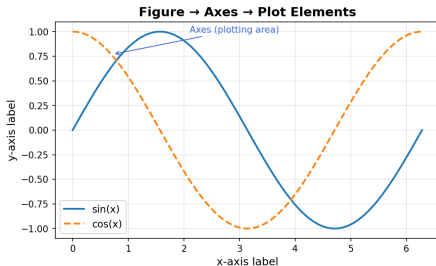
April 25, 2026

1. Data Visualization with Matplotlib
2. Data Visualization with Seaborn
3. Exploratory Data Analysis (EDA)
4. Data Processing: Scaling & Normalization
5. Train / Test Splitting
6. Feature Engineering

- ▶ **Understand your data** before building any model
 - Spot patterns, trends, and anomalies at a glance
 - Detect outliers and missing values early
- ▶ **Communicate results** clearly to others
- ▶ **Matplotlib** is Python's foundational plotting library
 - First released in 2003 (John Hunter)
 - Inspired by MATLAB's plotting interface
 - Nearly every other Python viz library builds on it

- ▶ **Figure:** the entire canvas / window
- ▶ **Axes:** a single plot area inside the figure
 - One figure can hold many axes (subplots)
- ▶ **Plot elements:** lines, markers, labels, legends, grid
- ▶ Two APIs:
 - `plt.plot(...)` — quick, MATLAB-style
 - `fig, ax = plt.subplots()` — explicit, recommended

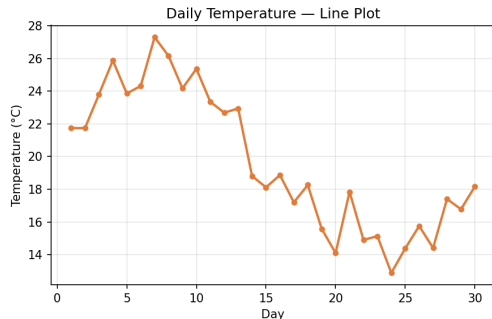
Figure (outer canvas)



- ▶ Best for **continuous data** over an ordered axis (e.g. time)
- ▶ Key parameters:
 - color, linestyle, linewidth
 - marker, markersize
 - label (for legend)

Code

```
ax.plot(days, temp,  
marker='o',  
color='#e07b39')  
ax.set_xlabel('Day')  
ax.set_title('Temperature')
```



- Shows **relationship between two numeric variables**
- Use color to encode a third variable (e.g. class label)

Per-class coloring

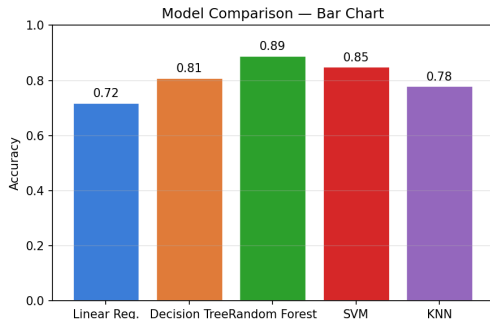
```
mask = y > 35  
ax.scatter(x[~mask],  
           y[~mask],  
           c='#3b7dd8',  
           label='Class A')  
ax.scatter(x[mask],  
           y[mask],  
           c='#e07b39',  
           label='Class B')  
ax.legend()
```



- ▶ **Compare values across categories**
- ▶ Variants:
 - Vertical (`ax.bar`) vs. horizontal (`ax.barh`)
 - Grouped bars, stacked bars
- ▶ Tip: annotate bars with their values for clarity

Code

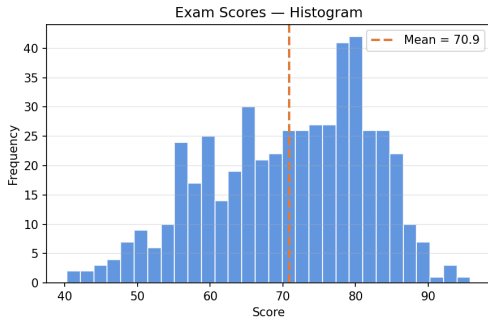
```
ax.bar(models, scores,  
color=[...],  
edgecolor='w')
```



- ▶ Shows the **distribution** of a single variable
- ▶ Bins data into intervals and counts occurrences
- ▶ Key parameters:
 - bins — number of intervals
 - edgecolor, alpha
- ▶ Overlay a vertical line for mean / median

Code

```
ax.hist(scores,  
bins=30)  
ax.axvline(scores.mean(),  
color='orange',  
ls='--')
```



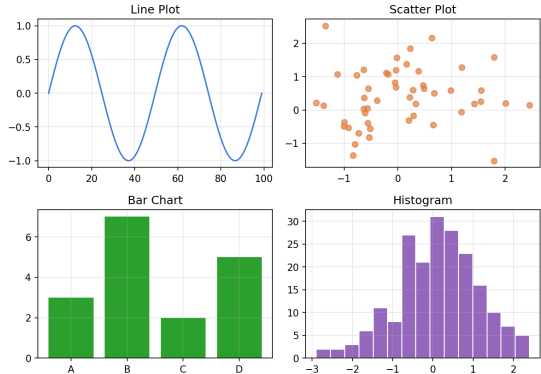
Subplots: Multiple Plots in One Figure

- ▶ `fig, axes = plt.subplots(rows, cols)`
- ▶ Returns a grid of Axes objects
- ▶ Use `axes[i, j]` to access each panel
- ▶ `fig.suptitle(...)` for an overall title
- ▶ `fig.tight_layout()` prevents overlapping labels

Code

```
fig, axes =  
plt.subplots(2,  
2)  
axes[0,0].plot(...)  
axes[0,1].scatter(...)
```

plt.subplots(2, 2) — Multiple Plots in One Figure



- ▶ A **high-level** Python visualization library built on top of Matplotlib
- ▶ Designed for **statistical graphics** with less code
- ▶ Advantages over raw Matplotlib:
 - Beautiful default themes and color palettes
 - Automatic aggregation and statistical summaries
 - Tight integration with **Pandas DataFrames**
 - Built-in support for categorical and distribution plots
- ▶ Import convention:

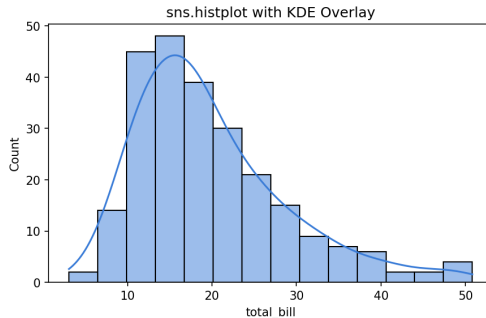
Setup

```
import seaborn as sns  
sns.set_theme(style='whitegrid')
```

- ▶ `sns.histplot()` — histogram with optional KDE overlay
- ▶ **KDE** (Kernel Density Estimation): smooth estimate of the probability density
- ▶ Key parameters:
 - `kde=True` to add the curve
 - `bins, stat ('count', 'density', 'probability')`
 - `hue` to split by category

Code

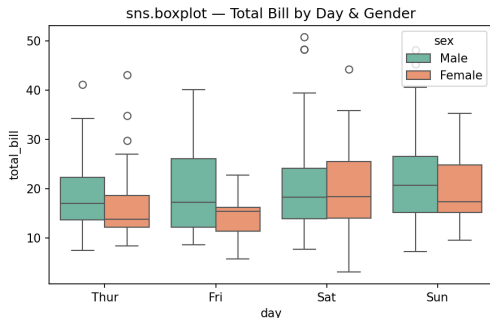
```
sns.histplot(data=tips,  
x='total_bill',  
kde=True)
```



- ▶ Shows **median**, **Q1/Q3** (box), **whiskers**, and **outliers**
- ▶ Great for comparing distributions across categories
- ▶ Key parameters:
 - `x` = categorical, `y` = numeric
 - `hue` to split by a second category

Code

```
sns.boxplot(data=tips,  
x='day',  
y='total_bill',  
hue='sex')
```

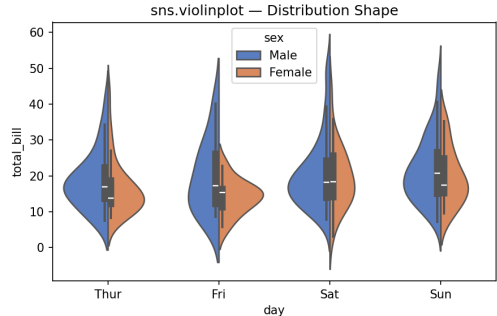


Violin Plot — Distribution Shape

- ▶ Combines a **boxplot** with a **KDE** on each side
- ▶ Reveals the full distribution shape, not just summary statistics
- ▶ `split=True`: show two categories mirrored for comparison
- ▶ When to prefer over boxplot:
 - Multimodal distributions
 - Need to see density, not just quartiles

Code

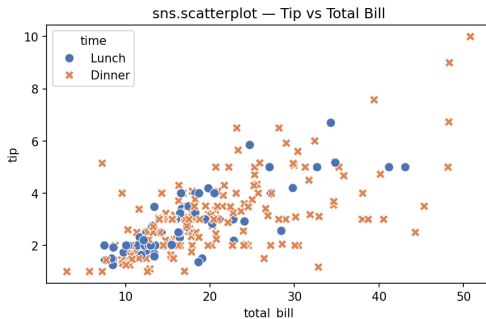
```
sns.violinplot(data=tips,  
x='day',  
y='total_bill',  
hue='sex',  
split=True)
```



- ▶ `sns.scatterplot()` adds **automatic legends** for hue, style, and size
- ▶ Encode up to **5 dimensions** in a 2D plot:
 - x, y positions
 - color (hue)
 - marker shape (style)
 - marker size (size)

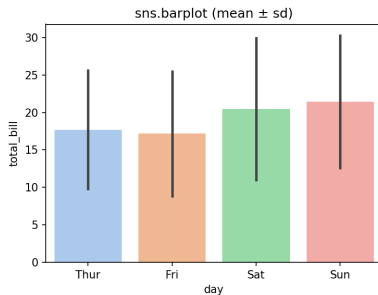
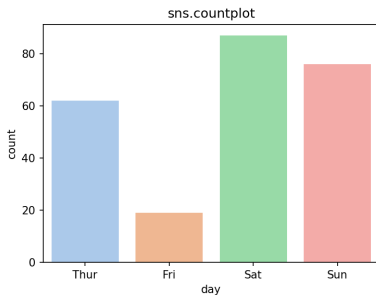
Code

```
sns.scatterplot(data=tips,  
x='total_bill',  
y='tip',  
hue='time',  
style='time')
```



Categorical Plots: countplot and barplot

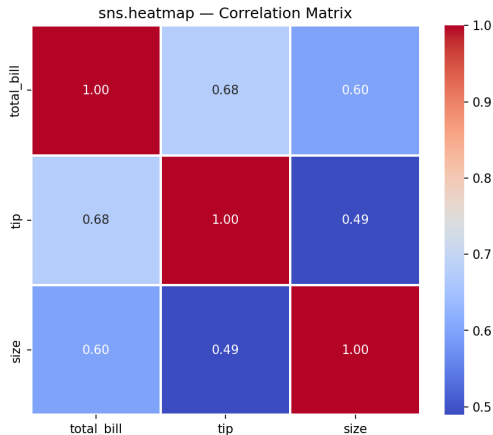
- ▶ `sns.countplot()` — counts occurrences of each category (like a histogram for categories)
- ▶ `sns.barplot()` — shows the **mean** (or other estimator) with error bars
 - Error bars show **confidence interval** by default (or `errorbar='sd'`)



- ▶ Visualize pairwise **correlations** between numeric features
- ▶ Colors encode correlation strength: -1 to $+1$
- ▶ Key parameters:
 - `annot=True` — show values
 - `cmap` — color palette
 - `square=True` — square cells
- ▶ Quickly identifies which features are **strongly correlated**

Code

```
corr = df.corr()  
sns.heatmap(corr,  
annot=True,  
cmap='coolwarm')
```

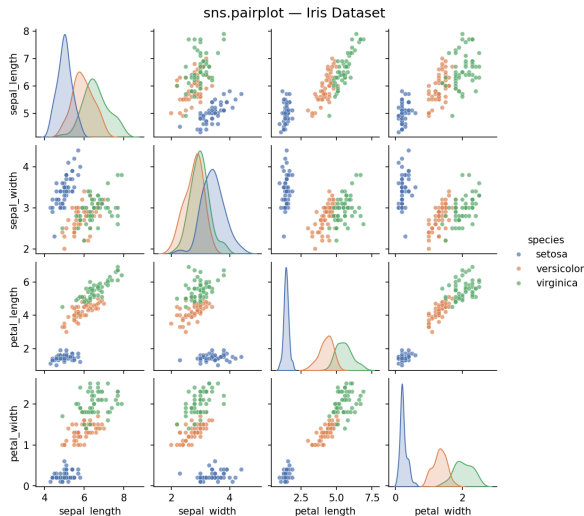


Pair Plot — All Feature Relationships at Once

- ▶ `sns.pairplot()` creates a grid of scatter plots for every pair of features
- ▶ Diagonal shows each feature's distribution
- ▶ hue colors points by class
- ▶ Extremely useful for:
 - Quick overview of **all relationships**
 - Checking **class separability**

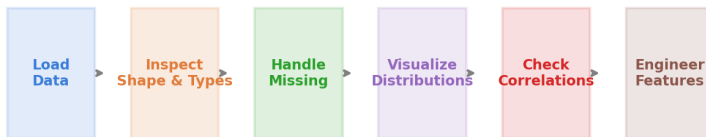
Code

```
sns.pairplot(iris,  
hue='species')
```



- ▶ **EDA** = systematically exploring your dataset *before* modeling
- ▶ Goal: understand the data's structure, quality, and relationships

Exploratory Data Analysis (EDA) Workflow



- ▶ Each step often reveals issues that change your approach
- ▶ **Iterate:** visualization → question → transformation → re-visualize

Step 1: Inspect Shape, Types, and Summary

- ▶ Start with Pandas basics before any plotting:

Essential Pandas Commands

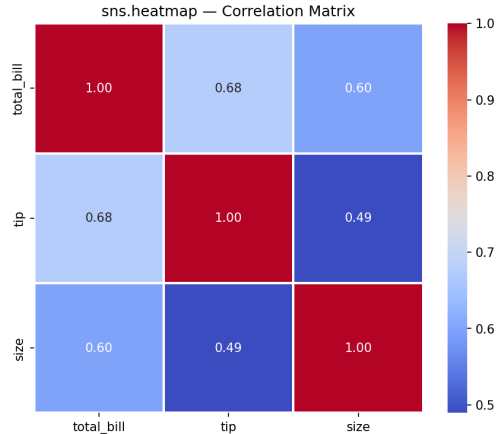
```
df.shape — number of rows and columns  
df.dtypes — data types per column  
df.describe() — mean, std, min, quartiles, max  
df.info() — non-null counts and memory  
df.isnull().sum() — missing values per column  
df.nunique() — unique values per column
```

- ▶ These tell you: **how big** is the data, **what types** are there, **how much** is missing

- ▶ For each numeric feature, check its distribution:
 - **Histogram / KDE:** Is it normal, skewed, multimodal?
 - **Boxplot:** Are there outliers?
- ▶ For each categorical feature:
 - **Countplot / value_counts():** Are classes balanced?
- ▶ **Why it matters:**
 - Skewed features → may need log or power transform
 - Outliers → may need robust scaling or removal
 - Imbalanced classes → may need resampling strategies

Step 3: Check Correlations and Relationships

- ▶ **Heatmap** of `df.corr()` reveals:
 - Strongly correlated features (redundancy)
 - Features correlated with target (useful predictors)
- ▶ **Pair plot** for small datasets:
 - See clusters and class separability
 - Spot non-linear relationships
- ▶ **Scatter plots** for specific pairs of interest



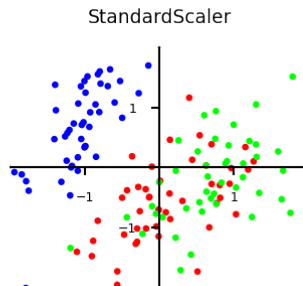
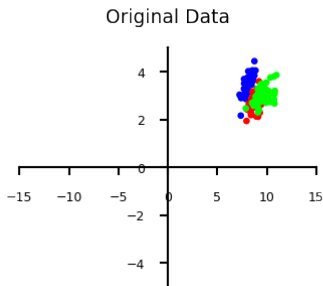
Which Plot Should I Use?

Goal	Recommended Plot
Distribution of one variable	Histogram, KDE, Boxplot
Relationship between two variables	Scatter plot, Line plot
Compare categories	Bar chart, Boxplot, Violin
Composition / proportions	Pie chart, Stacked bar
Correlation across features	Heatmap, Pair plot
Time series / trends	Line plot

- **Rule of thumb:** start simple (histogram, scatter), then layer complexity as needed

Why Scale Features?

- ▶ Different features often have **very different ranges**
 - e.g. age (0–100) vs. income (0–1,000,000)
- ▶ Features with larger values can **dominate** the model
- ▶ Goal: bring all features to a **comparable scale**



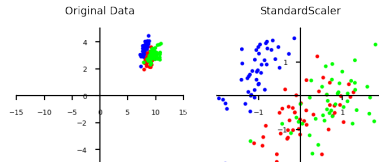
Standardization (Z-score Scaling)

- ▶ Subtract the mean μ , divide by standard deviation σ
- ▶ **Output range:** unbounded (typically -3 to $+3$)
- ▶ **When to use:** general-purpose default; assumes roughly normal distribution

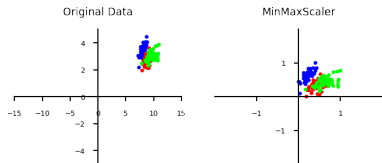
$$x_{\text{new}} = \frac{x - \mu}{\sigma}$$

Result

Mean becomes 0, standard deviation becomes 1.

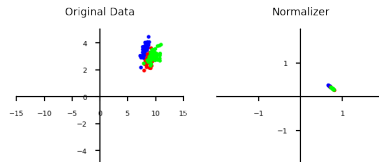


- ▶ Rescales values to a fixed range (usually $[0, 1]$)
- ▶ **Output range:** $[0, 1]$ (or any $[\min, \max]$)
- ▶ **When to use:** when you need bounded values (e.g. pixel intensities, neural network inputs)
- ▶ *Sensitive to outliers* — one extreme value stretches the whole scale



$$X_{\text{new}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- ▶ Scales each **row** (sample) so its values sum to a fixed norm
- ▶ **L1 norm:** values in each row sum to 1
- ▶ **L2 norm:** sum of squares in each row equals 1
- ▶ **Output range:** $[0, 1]$ per row (L1); depends on dimension (L2)
- ▶ **When to use:** text data (word counts), comparing samples by proportion rather than magnitude

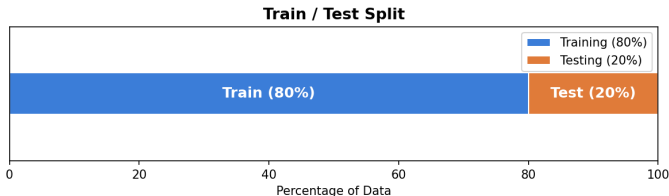


$$\text{L1: } x_{\text{new}} = \frac{x}{\sum |x_i|} \quad \text{L2: } x_{\text{new}} = \frac{x}{\sqrt{\sum x_i^2}}$$

Method	Formula	Output Range	Best For
Standardization	$(x - \mu) / \sigma$	Unbounded (≈ -3 to $+3$)	General default
Min-Max	$(x - x_{\min}) / (x_{\max} - x_{\min})$	$[0, 1]$	Bounded inputs
L1 Normalization	$x / \sum x_i $	Row sums to 1	Count / text data
L2 Normalization	$x / \sqrt{\sum x_i^2}$	Unit vector per row	Distance-based models

- **Standardization vs. Min-Max:** standardization is more robust to outliers
- Always fit the scaler on **training data only**, then apply to test data

- ▶ A model can **memorize** the training data instead of learning patterns
- ▶ We need a separate **test set** the model has never seen to check if it truly learned
- ▶ Typical split: **80% training, 20% testing**



- ▶ **Golden rule:** never look at or use the test set during training

- ▶ Simply take a percentage of the data and set it aside
- ▶ **Problem:** if data is ordered (e.g. labels sorted), the split can be biased
 - Labels: [1, 1, 1, 2, 2, 2, 3, 3, 3, 3]
 - Last 30% as test: [3, 3, 3] — classes 1 and 2 are missing!
- ▶ **Solution:** always **shuffle** before splitting

Code

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=True, random_state=42)
```

- ▶ Shuffling helps, but with **imbalanced classes** we can still get unlucky
 - e.g. 95% class A, 5% class B — test set might have 0 samples of class B
- ▶ **Stratification** ensures each split has the **same class proportions** as the full dataset

Code

```
X_train, X_test, y_train, y_test = train_test_split(  
X, y, test_size=0.2, stratify=y, random_state=42)
```

- ▶ If full data is 95%/5%, then train *and* test will both be $\approx$ 95%/5%
- ▶ **Rule of thumb:** always use `stratify=y` for classification tasks

- ▶ **Problem:** with small datasets, a single 80/20 split may not be representative
- ▶ **K-Fold** splits data into k equal parts (folds)
 - Train on $k-1$ folds, test on the remaining fold
 - Repeat k times, rotating the test fold
 - Average the k scores
- ▶ Every sample gets to be in the test set **exactly once**
- ▶ Common choice: $k = 5$ or $k = 10$



- ▶ Same idea as K-Fold, but each fold preserves the **class distribution**
- ▶ Combines the benefits of K-Fold (robust evaluation) with stratification (balanced classes)

Code

```
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5, shuffle=True)
for train_idx, test_idx in skf.split(X, y):
    X_train, X_test = X[train_idx], X[test_idx]
    # train model, compute score ...
```

- ▶ **Default choice** for classification — use this unless you have a reason not to

- ▶ Sometimes samples are **not independent** — they belong to groups
 - e.g. multiple medical images from the **same patient**
 - e.g. several transactions from the **same user**
- ▶ If the same patient appears in both train and test, the model can “cheat”
- ▶ **Group K-Fold** ensures all samples from one group are in the **same fold**

Code

```
from sklearn.model_selection import GroupKFold
gkf = GroupKFold(n_splits=5)
for train_idx, test_idx in gkf.split(X, y,
groups=patient_ids):
    # no patient appears in both train and test
```

- ▶ **Feature engineering** = creating new columns from existing data to help the model learn better
- ▶ The raw data is rarely in the best format for a model
- ▶ Think of it as *translating domain knowledge into numbers*
- ▶ We have not covered model types yet — for now treat the model as a **black box** that takes numbers in and outputs a prediction

Key idea

Better features → better model accuracy — often more impactful than choosing a fancier model.

Example Dataset: Predicting House Prices

date_sold	bedrooms	sqft	neighborhood	condition	price
2024-03-15	3	1,200	Downtown	Good	450,000
2024-07-22	4	1,800	Suburbs	Excellent	620,000
2024-12-01	2	900	Downtown	Fair	310,000
2025-01-10	3	1,500	Rural	Good	280,000

- ▶ Models need **numbers** — but neighborhood, condition, and date_sold are not numeric
- ▶ Can we create **new features** that capture useful information the model cannot see directly?

Use Case 1: Extracting from Dates

- ▶ A raw date like 2024-03-15 is meaningless as a number
- ▶ But we can **extract** useful parts:
 - `month` → captures seasonal trends (e.g. spring vs. winter sales)
 - `year` → captures market trends over time
 - `day_of_week` → weekend vs. weekday effects

Code

```
df['month'] = df['date_sold'].dt.month  
df['year'] = df['date_sold'].dt.year
```

<code>date_sold</code>	<code>month</code>	<code>year</code>
2024-03-15	3	2024
2024-07-22	7	2024
2024-12-01	12	2024

- ▶ Sometimes the **relationship** between features matters more than each feature alone
- ▶ Example: $\text{price_per_sqft} = \text{price} / \text{sqft}$
 - A 1,200 sqft house at \$450k and a 1,800 sqft house at \$620k look different
 - But \$375/sqft vs. \$344/sqft tells a clearer story
- ▶ Example: $\text{total_rooms} = \text{bedrooms} + \text{bathrooms}$

Code

```
df['price_per_sqft'] = df['price'] / df['sqft']  
df['total_rooms']    = df['bedrooms'] + df['bathrooms']
```

- ▶ **Ratios**, **sums**, and **differences** between columns are simple but powerful features

Use Case 3: Creating Flags / Indicators

- ▶ Turn domain knowledge into **binary (0/1) flags**
- ▶ Example: `is_large = 1` if `sqft > 1,500`, else 0
- ▶ Example: `is_downtown = 1` if `neighborhood == 'Downtown'`, else 0
- ▶ These help the model distinguish **meaningful categories** you already know about

Code

```
df['is_large']      = (df['sqft'] > 1500).astype(int)
df['is_downtown'] = (df['neighborhood'] ==
                    'Downtown').astype(int)
```

sqft	neighborhood	is_large	is_downtown
1,200	Downtown	0	1
1,800	Suburbs	1	0
900	Downtown	0	1

- ▶ condition has a natural order: Fair < Good < Excellent
- ▶ **Ordinal encoding** maps categories to integers preserving that order

condition	condition_encoded
Fair	0
Good	1
Excellent	2

- ▶ **When to use:** the categories have a meaningful order (low / medium / high, small / large, etc.)
- ▶ **Caution:** the model will assume 2 is “twice” 1 — only use when that makes sense

- ▶ neighborhood has no natural order (Downtown, Suburbs, Rural)
- ▶ **One-hot encoding** creates a separate binary column for each category

neighborhood	is_Downtown	is_Suburbs	is_Rural
Downtown	1	0	0
Suburbs	0	1	0
Downtown	1	0	0
Rural	0	0	1

Code

```
pd.get_dummies(df, columns=['neighborhood'])
```

- ▶ **When to use:** categories have no order (colors, cities, product types)
- ▶ **Caution:** many unique values → many columns (high dimensionality)

Credits

KAUST Academy

King Abdullah University of Science and Technology

KAUST–Mawhiba IOAI 2026 Training Program